

AI / ML Basic Concepts Overview

SECTION 1: FUNDAMENTAL AI CONCEPTS

1. What is Artificial Intelligence?

Best Answer: "Artificial Intelligence is the simulation of human intelligence in machines. It enables computers to perform tasks that typically require human intelligence like learning, reasoning, problem-solving, and decision-making."

Real Examples:

- Voice assistants (Siri, Alexa)
- Recommendation systems (Netflix, YouTube)
- Face recognition
- Self-driving cars
- Game-playing AI (chess, Go)

Follow-up Q: Types of AI?

- **Narrow AI:** Designed for specific tasks (face recognition, spam filter)
 - **General AI:** Human-level intelligence across all tasks (doesn't exist yet)
-

2. What is Machine Learning?

Best Answer: "Machine Learning is a subset of AI that enables computers to learn and improve from experience without being explicitly programmed. The machine learns patterns from data."

Key Point: Traditional programming = we give rules, ML = machine discovers rules from data

Example:

- Traditional: "If email contains 'win lottery', mark spam"
- ML: Show 10,000 spam emails, let it learn patterns automatically

Viva Q&A:

- **Q: AI vs ML difference?**

- A: AI is the broader goal of making intelligent machines. ML is a specific method to achieve AI by learning from data.
-

3. What is Deep Learning?

Best Answer: "Deep Learning is a subset of ML that uses artificial neural networks with multiple layers (hence 'deep'). It's especially powerful for image recognition, speech, and language tasks."

Hierarchy: AI $\supset$ ML $\supset$ Deep Learning

SECTION 2: TYPES OF LEARNING (VERY IMPORTANT)

SUPERVISED LEARNING

Definition: Learning from labeled data where we know the correct answers.

How it works:

1. Give model input + correct output
2. Model learns the mapping
3. Can predict outputs for new inputs

Formula: Input (X) $\rightarrow$ Model learns $\rightarrow$ Output (Y)

- We have both X and Y during training

Examples:

- **Classification:** Email $\rightarrow$ Spam/Not Spam
- **Regression:** House features $\rightarrow$ Price

Common Algorithms:

- Linear Regression
- Logistic Regression
- Decision Trees

- KNN
- Support Vector Machines (SVM)
- Neural Networks

Real-life Example: Teaching a child with flashcards:

- Show picture of cat + label "CAT" ✓
- Show picture of dog + label "DOG" ✓
- Child learns to recognize them

UNSUPERVISED LEARNING

Definition: Learning from unlabeled data where we don't know the correct answers. Model finds hidden patterns on its own.

How it works:

1. Give model only input data (no labels)
2. Model discovers structure/patterns
3. Groups similar data together

Formula: Input (X) → Model finds patterns → Discovers groups

- We only have X, no Y

Examples:

- Customer segmentation (grouping similar customers)
- Topic detection in documents
- Anomaly detection

Common Algorithms:

- K-means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)

Real-life Example: Give child mixed toys without labels:

- Child groups similar toys together
- Cars in one pile, dolls in another
- Child discovers categories on their own

REINFORCEMENT LEARNING 🎮

Definition: Learning by trial and error through rewards and penalties.

How it works:

1. Agent takes action in environment
2. Gets reward (good) or penalty (bad)
3. Learns to maximize rewards over time

Examples:

- Game-playing AI (AlphaGo, chess)
- Robot learning to walk
- Self-driving cars

Real-life Example: Training a dog:

- Dog sits → Give treat (reward) 🍖
- Dog jumps on furniture → Scold (penalty) ❌
- Dog learns which actions get treats

COMPARISON TABLE (MEMORIZE THIS)

Aspect	Supervised	Unsupervised	Reinforcement
Data	Labeled (X, Y)	Unlabeled (X only)	Environment + feedback
Goal	Predict Y from X	Find patterns in X	Maximize rewards
Example	Email classification	Customer grouping	Game playing
Teacher	Yes (labels)	No	Reward/penalty signal

SECTION 3: OVERFITTING VS UNDERFITTING (SUPER IMPORTANT)

UNDERFITTING

Definition: Model is too simple and doesn't learn the data well.

Characteristics:

- High training error
- High testing error
- Model is too basic

Real-life Example 1: You're learning to identify animals. Your only rule: "Has 4 legs = Dog"

- Problem: Cats, horses, tables also have 4 legs!
- Too simple, doesn't capture the pattern

Real-life Example 2: Studying for exam by only reading chapter titles

- You don't learn enough details
- Fail both practice tests and actual exam

Signs:

- Low accuracy on training data
- Low accuracy on test data
- Model doesn't fit data well

OVERFITTING

Definition: Model learns training data TOO well, including noise and outliers. Doesn't generalize to new data.

Characteristics:

- Very low training error
- High testing error
- Model memorizes instead of learning

Real-life Example 1: You memorize every single dog photo you've seen, including background, lighting, photographer's name

- Problem: See a dog in different setting → don't recognize it
- Too specific, can't generalize

Real-life Example 2: Studying for exam by memorizing exact practice problems word-for-word

- You ace practice tests
- Fail actual exam because questions are phrased differently

Signs:

- Very high accuracy on training data (98-100%)
 - Low accuracy on test data
 - Large gap between training and test performance
-

GOOD FIT (THE GOAL) 

Characteristics:

- Moderate training error
- Similar testing error
- Model generalizes well

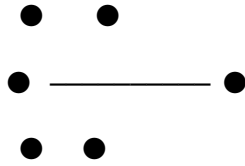
How to achieve:

- Right model complexity
 - Enough (but not too much) training
 - Proper validation
-

VISUAL ANALOGY

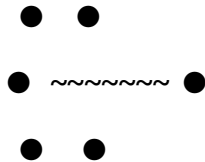
Imagine fitting a line to scattered data points:

Underfitting:



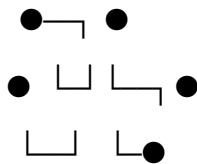
Straight line through curved data - too simple!

Good Fit:



Smooth curve following the trend - just right!

Overfitting:



Zigzag touching every point - too complex!

HOW TO PREVENT OVERFITTING (Common Viva Question!)

1. **More training data** - Harder to memorize more examples
 2. **Simpler model** - Fewer parameters to overfit with
 3. **Regularization** - Add penalty for complex models
 4. **Cross-validation** - Test on multiple data splits
 5. **Early stopping** - Stop training before overfitting occurs
 6. **Dropout** (in neural networks) - Randomly disable neurons during training
-

HOW TO PREVENT UNDERFITTING

1. **More complex model** - Add more features/parameters
2. **Train longer** - Give model more time to learn

3. **Better features** - Use more informative input data
 4. **Reduce regularization** - If too much constraint
-

SECTION 4: TRAINING VS TESTING DATA

Training Data (70-80% of total data)

Purpose: Teach the model

What happens:

- Model learns patterns
- Adjusts its parameters
- Sees these examples multiple times

Analogy: Practice problems you solve while studying

Testing Data (20-30% of total data)

Purpose: Evaluate the model on unseen data

What happens:

- Model never sees this during training
- Used only once at the end
- Measures real-world performance

Analogy: Actual exam questions (different from practice)

Validation Data (Optional, 10-15%)

Purpose: Fine-tune model during training

What happens:

- Used to check performance during training
- Helps prevent overfitting
- Guides hyperparameter tuning

Analogy: Mock tests before final exam

WHY SPLIT DATA? (Important Viva Question!)

Q: Why not use all data for training?

A: If we train and test on same data, we can't tell if model truly learned patterns or just memorized. It's like taking the exact same exam you practiced on - doesn't prove real understanding.

Q: What if testing accuracy is much lower than training accuracy?

A: This indicates overfitting. Model memorized training data but can't generalize.

SECTION 5: CLASSIFICATION VS REGRESSION

CLASSIFICATION 📊

Definition: Predicting a category/class label

Output: Discrete values (categories)

Examples:

- Email: Spam or Not Spam
- Image: Cat, Dog, or Bird
- Disease: Present or Absent
- Customer: Will Buy or Won't Buy

Types:

- **Binary:** 2 classes (Yes/No, 0/1)
- **Multi-class:** 3+ classes (Cat/Dog/Bird)

Common Algorithms:

- Logistic Regression
- KNN
- Decision Trees

- SVM
- Neural Networks

REGRESSION

Definition: Predicting a continuous numerical value

Output: Real numbers

Examples:

- House price: \$250,000
- Temperature: 28.5°C
- Stock price: \$145.67
- Student score: 85.3

Common Algorithms:

- Linear Regression
- Polynomial Regression
- Decision Trees
- Neural Networks

KEY DIFFERENCE

Aspect	Classification	Regression
Output	Category/Label	Number
Example	"Spam"	\$250,000

Evaluation Accuracy, Precision, Recall MSE, RMSE, R^2

Viva Tip: Think: "Class = Category, Regression = Real number"

SECTION 6: MODEL EVALUATION METRICS

ACCURACY ✓

Definition: Percentage of correct predictions

Formula:

$$\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions}) \times 100$$

Example:

- Model predicts 100 emails
- 90 are correct
- Accuracy = $90/100 = 90\%$

When good: Balanced datasets

Limitation: Not good for imbalanced data

- If 95% emails are not spam, saying "everything is not spam" gives 95% accuracy but is useless!

LOSS/ERROR ✗

Definition: How wrong the model's predictions are

Key Points:

- Lower loss = better model
- During training, we minimize loss
- Different types: MSE, Cross-Entropy, etc.

Example:

- Predict house price: \$300,000
- Actual price: \$280,000
- Error: \$20,000

CONFUSION MATRIX (For Classification)

Predicted

Yes No

Actual Yes TP FN

No FP TN

- **TP (True Positive):** Correctly predicted Yes
 - **TN (True Negative):** Correctly predicted No
 - **FP (False Positive):** Wrongly predicted Yes (Type I Error)
 - **FN (False Negative):** Wrongly predicted No (Type II Error)
-

PRECISION & RECALL

Precision: Of all predicted positive, how many are actually positive?

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

"How precise are positive predictions?"

Recall: Of all actual positive, how many did we find?

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

"How many positives did we recall/capture?"

Example:

- Cancer detection
 - **High Precision needed:** Don't want to scare healthy people (minimize FP)
 - **High Recall needed:** Don't want to miss cancer patients (minimize FN)
-

SECTION 7: ALGORITHMS FROM YOUR SYLLABUS

1. K-NEAREST NEIGHBORS (KNN)

Concept: "You are the average of your K closest friends"

How it works:

1. To classify new point
2. Find K nearest points in training data

3. Take majority vote of their labels

Example: Want to predict if you'll like a movie:

- Find 5 people most similar to you ($K=5$)
- 4 liked it, 1 didn't
- Prediction: You'll like it

Viva Q&A:

- **Q: How does KNN work?**
 - A: KNN classifies a data point by finding the K nearest neighbors in the training set and assigning the most common label among them.
 - **Q: What is K in KNN?**
 - A: K is the number of nearest neighbors to consider. It's a hyperparameter we choose.
 - **Q: How to choose K ?**
 - A: Too small K (like 1) $\rightarrow$ overfitting, too large $K \rightarrow$ underfitting. Usually use odd numbers (3, 5, 7) and test multiple values.
-

2. K-MEANS CLUSTERING

Type: Unsupervised learning

Purpose: Group similar data into K clusters

How it works:

1. Choose K (number of clusters)
2. Randomly place K centroids
3. Assign each point to nearest centroid
4. Recalculate centroids as cluster means
5. Repeat steps 3-4 until convergence

Example: Customer segmentation:

- Group customers by buying behavior

- Cluster 1: High spenders
- Cluster 2: Occasional buyers
- Cluster 3: Window shoppers

Viva Q&A:

- **Q: What is K-means?**
 - A: An unsupervised algorithm that partitions data into K clusters by minimizing the distance between points and their cluster centroid.
 - **Q: How to choose K?**
 - A: Use elbow method - plot number of clusters vs. total distance, look for "elbow" point where adding more clusters doesn't help much.
-

3. DECISION TREES

Concept: Series of yes/no questions to reach a decision

How it works:

- Split data based on features
- Choose best feature to split on
- Repeat until pure groups or stopping criteria

Example: Should I play tennis today?

Outlook?

└─ Sunny → Humidity?

| └─ High → No

| └─ Normal → Yes

└─ Overcast → Yes

└─ Rainy → Wind?

└─ Strong → No

└─ Weak → Yes

Advantages:

- Easy to understand and visualize
- No data preparation needed
- Handles both numerical and categorical data

Disadvantages:

- Can overfit easily
 - Small changes in data can change tree
-

4. LINEAR REGRESSION

Purpose: Predict continuous values by fitting a line

Formula: $y = mx + b$

- y = prediction
- x = input
- m = slope (weight)
- b = intercept (bias)

Example: Predict house price based on size:

- Size = 1000 sq ft → Price = \$200,000
- Size = 2000 sq ft → Price = \$350,000
- Learn: Price = \$150 × Size + \$50,000

Viva Q&A:

- **Q: What is linear regression?**
 - A: A supervised learning algorithm that models the relationship between input and output as a straight line, used for predicting continuous values.
-

SECTION 8: NEURAL NETWORKS & DEEP LEARNING**ARTIFICIAL NEURAL NETWORK (ANN)**

Concept: Inspired by human brain neurons

Components:

1. **Input Layer:** Receives data
2. **Hidden Layer(s):** Process information
3. **Output Layer:** Gives prediction

Neuron Structure:

Inputs → Weights → Summation → Activation → Output

Example: Predicting house price:

- Inputs: size, bedrooms, location
 - Hidden layers process these
 - Output: predicted price
-

HOW NEURAL NETWORKS LEARN

Forward Pass (Forward Propagation)

1. Input enters network
2. Data flows through layers
3. Each neuron: weighted sum + activation
4. Produces output prediction

Backward Pass (Backpropagation)

1. Compare prediction with actual answer
2. Calculate error/loss
3. Propagate error backward through network
4. Adjust weights to reduce error
5. Repeat for many iterations

Analogy:

- Forward pass: Student takes exam

- Calculate loss: Teacher checks answers
 - Backpropagation: Teacher shows mistakes
 - Adjust weights: Student learns from mistakes
-

ACTIVATION FUNCTIONS (Common Question!)

Purpose: Introduce non-linearity (ability to learn complex patterns)

Common Types:

1. **Sigmoid:** Output between 0 and 1
 - Used for binary classification
 - Problem: Vanishing gradient
2. **ReLU (Rectified Linear Unit):**
 - $f(x) = \max(0, x)$
 - Most popular in hidden layers
 - Fast and effective
3. **Softmax:**
 - Used in output layer for multi-class
 - Converts to probabilities (sum = 1)

Viva Q&A:

- **Q: Why activation functions?**
 - A: Without activation, neural network is just linear algebra - can't learn complex patterns. Activation adds non-linearity.
 - **Q: Most common activation?**
 - A: ReLU for hidden layers, Sigmoid for binary output, Softmax for multi-class output.
-

BACKPROPAGATION (Important!)

Definition: Algorithm to train neural networks by propagating errors backward

Steps:

1. Forward pass: Make prediction
2. Calculate loss: Compare with actual
3. Backward pass: Calculate gradients (how much each weight contributed to error)
4. Update weights: Move in direction that reduces error
5. Repeat

Key Point: Uses chain rule from calculus to find gradients

Viva Q&A:

- **Q: What is backpropagation?**
 - A: The algorithm that adjusts weights in a neural network by calculating gradients of the loss function and propagating them backward from output to input layers.
 - **Q: Why is it called backpropagation?**
 - A: Because the error is propagated backward through the network layers to update weights.
-

GRADIENT DESCENT

Concept: Method to find minimum of loss function

How it works:

1. Start with random weights
2. Calculate gradient (slope) of loss
3. Move weights in opposite direction of gradient
4. Take small steps (learning rate)
5. Repeat until convergence

Analogy: You're blindfolded on a mountain, trying to reach valley:

- Feel the slope (gradient)
- Take small steps downhill

- Eventually reach lowest point (minimum loss)

Learning Rate:

- Too large: Miss minimum, oscillate
 - Too small: Takes forever
 - Just right: Converges efficiently
-

RECURRENT NEURAL NETWORKS (RNN)

Special Feature: Has memory of previous inputs

Why needed: For sequential data where order matters

- Text: "I love this" vs "This love I"
- Time series: Yesterday's stock price affects today's

How it works:

- Each step receives current input + hidden state from previous step
- Hidden state = memory

Examples:

- Language translation
- Speech recognition
- Text generation
- Stock price prediction
- Video analysis

Viva Q&A:

- **Q: When to use RNN?**
 - A: When data has temporal or sequential dependencies where previous information is relevant to current prediction.
- **Q: Difference between ANN and RNN?**

- A: ANN treats each input independently. RNN has recurrent connections that allow it to maintain memory of previous inputs, making it suitable for sequences.
-

CONVOLUTIONAL NEURAL NETWORKS (CNN)

Special Feature: Designed for image/spatial data

Why needed: Images have spatial relationships

- Nearby pixels are related
- Patterns repeat (edges, textures)

Key Concepts:

- **Convolution:** Slide filter over image to detect features
- **Pooling:** Reduce size while keeping important features
- **Filters:** Learn to detect edges, shapes, objects

Examples:

- Image classification
- Face recognition
- Object detection
- Medical image analysis

Viva Q&A:

- **Q: When to use CNN?**
 - A: For image, video, or any data with spatial relationships where local patterns are important.
-

SECTION 9: SEARCH ALGORITHMS

BREADTH-FIRST SEARCH (BFS)

Strategy: Explore all neighbors at current level before going deeper

Uses: Queue (FIFO - First In First Out)

How it works:

1. Start from root
2. Visit all children
3. Then visit all grandchildren
4. Continue level by level

Example:

A

/\

B C D

/| \

E F G H

Order: A → B,C,D → E,F,G,H

Properties:

- Finds shortest path
 - Complete (finds solution if exists)
 - Uses more memory (stores all nodes at current level)
-

DEPTH-FIRST SEARCH (DFS)

Strategy: Explore one path completely before backtracking

Uses: Stack (LIFO - Last In First Out)

How it works:

1. Start from root
2. Go deep down one path
3. When stuck, backtrack
4. Try next path

Example:

A

/\

B C D

/| \

E F G H

Order: A → B → E → F → (back to B) → C → G → H → D

Properties:

- Uses less memory
- May not find shortest path
- Good for exploring all possibilities

BFS VS DFS COMPARISON

Aspect	BFS	DFS
Data Structure	Queue	Stack
Memory	High (stores level)	Low (stores path)
Shortest Path	Yes ✓	No ✗
When to use	Shortest path needed Memory limited, explore all	

Viva Q&A:

- **Q: Main difference?**
 - A: BFS explores level-by-level using queue; DFS explores depth-first using stack.
- **Q: Which finds shortest path?**
 - A: BFS always finds shortest path; DFS may not.

A ALGORITHM*

Type: Informed search (uses heuristic)

Formula: $f(n) = g(n) + h(n)$

- $g(n)$ = cost from start to current node
- $h(n)$ = estimated cost from current to goal (heuristic)
- $f(n)$ = total estimated cost

Why better: Uses knowledge about goal to search smarter

Example: GPS navigation

- $g(n)$ = distance traveled
 - $h(n)$ = straight-line distance to destination
 - Finds optimal route efficiently
-

SECTION 10: GAME THEORY ALGORITHMS

MINIMAX ALGORITHM

Purpose: Find best move in two-player games

Concept:

- MAX player tries to maximize score
- MIN player tries to minimize score
- Assume both play optimally

How it works:

1. Build game tree (all possible moves)
2. Assign values to terminal states (win/lose/draw)
3. Propagate values up:
 - MAX chooses maximum value
 - MIN chooses minimum value
4. Root value = best move

Example: Tic-Tac-Toe

- Your turn (MAX): Choose move with highest value
- Opponent turn (MIN): Choose move with lowest value
- Tree shows all possible game states

Viva Q&A:

- **Q: What is Minimax?**
 - A: An algorithm for two-player zero-sum games that finds the optimal move by assuming both players play perfectly, with one maximizing and other minimizing the score.

ALPHA-BETA PRUNING

Purpose: Optimize Minimax by cutting unnecessary branches

Concept:

- Skip branches that won't affect final decision
- Same result as Minimax, but faster

How it works:

- **Alpha:** Best value MAX can guarantee
- **Beta:** Best value MIN can guarantee
- If $\text{Beta} \leq \text{Alpha}$ at any point, prune (stop exploring) that branch

Benefit:

- Reduces nodes evaluated
- Can search deeper in same time
- Same answer as Minimax

Example: If MAX already found a move giving score 5, and current branch can only give maximum 3, no need to explore it further.

Viva Q&A:

- **Q: Difference from Minimax?**

- A: Alpha-Beta gives same result as Minimax but is faster by pruning branches that won't affect the final decision.

SECTION 11: FUZZY LOGIC

WHAT IS FUZZY LOGIC?

Traditional Logic:

- Binary: True or False, Yes or No, 0 or 1

Fuzzy Logic:

- Deals with partial truth
- Values between 0 and 1
- Handles uncertainty and vagueness

WHY FUZZY LOGIC?

Real world is not black and white:

- "Is it hot?" → Not just yes/no, but "how hot?"
- "Is person tall?" → Depends on context

Membership Functions: Instead of:

- Temperature > 30°C = Hot (yes/no)

Fuzzy:

- 25°C → 0.3 hot (30% membership in "hot")
- 30°C → 0.7 hot (70% membership)
- 35°C → 1.0 hot (100% membership)

FUZZY SETS

Example: Age classification

Crisp Sets (Traditional):

- Young: 0-30
- Middle: 31-60
- Old: 61+

Problem: Is person aged 30 young or middle?

Fuzzy Sets:

- Age 25: 1.0 Young, 0.0 Middle
- Age 30: 0.7 Young, 0.3 Middle
- Age 35: 0.3 Young, 0.7 Middle

Allows gradual transition!

APPLICATIONS

1. Air Conditioning:

- Input: Room temperature
- Output: Cooling power
- Rules: "If temp is high, cooling is strong"

2. Washing Machine:

- Inputs: Dirt level, fabric type
- Output: Wash time
- Rules: "If very dirty and delicate, wash gently but longer"

3. Camera Auto-Focus:

- Inputs: Contrast, distance
- Output: Focus adjustment

Viva Q&A:

- **Q: What is fuzzy logic?**
 - A: A form of logic that handles partial truth values between 0 and 1, unlike traditional binary logic. Used when dealing with uncertainty and vague concepts.

- **Q: Why use fuzzy logic?**
 - A: Real-world concepts like "hot," "tall," "fast" are not binary. Fuzzy logic models human reasoning better by allowing degrees of truth.
-

SECTION 12: CONSTRAINT SATISFACTION PROBLEMS (CSP)

WHAT IS CSP?

Definition: Problems where we need to find values for variables that satisfy certain constraints.

Components:

1. **Variables:** Things to assign values to
 2. **Domains:** Possible values for each variable
 3. **Constraints:** Rules that must be satisfied
-

EXAMPLES

Sudoku:

- Variables: Empty cells
- Domain: Numbers 1-9
- Constraints:
 - Each row has unique numbers
 - Each column has unique numbers
 - Each 3×3 box has unique numbers

Map Coloring:

- Variables: Countries/states
- Domain: Colors (red, blue, green)
- Constraints: Adjacent regions can't have same color

Class Scheduling:

- Variables: Classes

- Domain: Time slots
 - Constraints:
 - No teacher in two places at once
 - Room capacity limits
 - No student schedule conflicts
-

SOLVING CSP

Methods:

1. **Backtracking:** Try values, backtrack if constraints violated
2. **Forward Checking:** Eliminate incompatible values
3. **Arc Consistency:** Ensure all pairs of variables are consistent

Viva Q&A:

- **Q: What is CSP?**
 - A: Constraint Satisfaction Problems involve finding assignments for variables that satisfy all given constraints. Examples include Sudoku, map coloring, and scheduling.
-

SECTION 13: QUICK FIRE VIVA QUESTIONS & ANSWERS

Concept Questions:

1. **Q: What's the goal of machine learning?**
 - A: To enable computers to learn patterns from data and make predictions without explicit programming.
2. **Q: Name 3 types of machine learning**
 - A: Supervised learning, Unsupervised learning, and Reinforcement learning.
3. **Q: What is a dataset?**
 - A: A collection of data used for training and testing machine learning models.
4. **Q: What are features?**

- A: Input variables or attributes used to make predictions (like house size, location for price prediction).

5. Q: What is a label?

- A: The output or target variable we want to predict (like house price, spam/not spam).

6. Q: What is a model?

- A: A mathematical representation learned from data that can make predictions on new inputs.

7. Q: What is training?

- A: The process of teaching a model by showing it examples and adjusting its parameters to minimize error.

8. Q: What is prediction?

- A: Using a trained model to estimate output for new, unseen input data.

9. Q: What is generalization?

- A: Model's ability to perform well on new, unseen data, not just training data.

10. Q: What is bias in ML?

- A: Error from overly simple assumptions. High bias leads to underfitting.

11. Q: What is variance in ML?

- A: Model's sensitivity to small fluctuations in training data. High variance leads to overfitting.

12. Q: What is bias-variance tradeoff?

- A: Balance between model being too simple (high bias) and too complex (high variance). Goal is to find sweet spot.

Algorithm Questions:

13. Q: When would you use KNN?

- A: For simple classification/regression when data is not too large and we want an intuitive, instance-based approach.

14. Q: Advantage of Decision Trees?

- A: Easy to understand, visualize, and interpret. No need for data normalization.

15. Q: Disadvantage of Decision Trees?

- A: Prone to overfitting, small data changes can create very different trees.

16. Q: When to use neural networks?

- A: For complex, non-linear problems with large datasets, especially image/speech/text tasks.

17. Q: Why use deep learning over traditional ML?

- A: Deep learning automatically learns features from raw data and performs better on complex tasks like image recognition, but requires more data and computation.

18. Q: What problem do RNNs solve?

- A: They handle sequential data by maintaining memory of previous inputs, crucial for time series, text, and speech.

Practical Questions:

19. Q: Your model has 99% training accuracy but 60% test accuracy. What's wrong?

- A: Overfitting. Model memorized training data but can't generalize.

20. Q: Your model has 65% training and 64% test accuracy. What's wrong?

- A: Underfitting. Model is too simple and hasn't learned the patterns well.

21. Q: How to improve a model?

- A: Depends on problem:
 - Underfitting: More complex model, more features, train longer
 - Overfitting: More data, simpler model, regularization, dropout

22. Q: Why normalize/scale data?

- A: Features with larger scales can dominate learning

Continue

Dec 18

. Normalization puts all features on similar scale for fair learning.

23. Q: What is feature engineering?

- A: Creating new features or transforming existing ones to improve model performance.

24. Q: What is hyperparameter tuning?

- A: Adjusting model settings (like K in KNN, learning rate) that aren't learned from data to optimize performance.

25. Q: What is cross-validation?

- A: Technique to assess model performance by splitting data into multiple folds, training on some, testing on others, and averaging results.

Comparison Questions:

26. Q: Supervised vs Unsupervised - which is better?

- A: Neither is "better" - depends on problem. Supervised when we have labels, unsupervised when we want to discover patterns.

27. Q: Deep Learning vs Traditional ML?

- A: Deep learning better for complex tasks with lots of data (images, speech). Traditional ML better for smaller datasets and interpretable results.

28. Q: BFS vs DFS - which is better?

- A: BFS for shortest path, DFS for memory efficiency. Context-dependent.

SECTION 14: YOUR PRACTICAL FILES - BE READY

Based on your GitHub (Image 4), know these:

1. Tictactoe.ipynb - Minimax implementation

- Be ready to explain game tree

- How minimax decides best move
 - 2. **AlphaBetaPruning_code_reproduce.ipynb** - Optimization
 - How pruning reduces computation
 - Same result, faster
 - 3. **A_Algorithm_Reproduced.ipynb** - A* search
 - Explain $f(n) = g(n) + h(n)$
 - Why it's better than BFS/DFS
 - 4. **KNN_Result_Reproduce.ipynb** - K-Nearest Neighbors
 - How it classifies
 - Effect of choosing different K
 - 5. **BackPropogation_REproduce.ipynb** - Neural network training
 - Forward and backward pass
 - How weights are updated
 - 6. **FuzzyReasoning_Reproduce.ipynb** - Fuzzy logic
 - Membership functions
 - Fuzzy rules
 - 7. **waterjug_problem_answers.ipynb** - Classic AI problem
 - State space search
 - Goal-reaching strategy
-

SECTION 15: LAST MINUTE MEMORIZATION CARD

Must Remember Definitions:

Term	One-Liner
AI	Making machines intelligent
ML	Learning from data automatically

Term	One-Liner
Supervised	Learning with labeled examples
Unsupervised	Finding patterns without labels
Overfitting	Memorizing training data, poor on test
Underfitting	Too simple, poor on both
Classification	Predicting categories
Regression	Predicting numbers
Neural Network	Brain-inspired layered model
Backpropagation	Adjusting weights by propagating errors backward
KNN	Classify by K nearest neighbors
K-means	Group data into K clusters
BFS	Level-by-level search using queue
DFS	Depth-first search using stack
Minimax	Game algorithm assuming optimal play
Fuzzy Logic	Logic with degrees of truth (0 to 1)